

Task 1: Exploratory Data Analysis on the Titanic Dataset

2. Objective

The objective is to learn how to perform **Exploratory Data Analysis (EDA)** on a CSV dataset using Python.

After completion, students should be able to:

- Load a CSV dataset into Python.
 - Inspect rows, columns, and data types.
 - Identify missing values.
 - Identify duplicate records.
 - Study unique values in columns.
 - Calculate basic statistical information.
 - Clean missing and duplicate data.
 - Perform simple group-based analysis.
 - Create histograms, boxplots, bar plots, and heatmaps.
 - Interpret patterns found during EDA.
-

3. Dataset

The **Titanic passenger dataset**.

The dataset contains information about passengers such as:

- **PassengerId** - unique passenger number.
- **Survived** - whether the passenger survived.
 - 0 = Did not survive
 - 1 = Survived
- **Pclass** - passenger class.
 - 1 = First class
 - 2 = Second class
 - 3 = Third class
- **Name** - passenger name.
- **Sex** - passenger gender.
- **Age** - passenger age.
- **SibSp** - number of siblings/spouses travelling with the passenger.

- **Parch** - number of parents/children travelling with the passenger.
 - **Ticket** - ticket number.
 - **Fare** - amount paid for the ticket.
 - **Cabin** - cabin number.
 - **Embarked** - port from which the passenger boarded.
-

Part A - Setting Up Python

Task 1: Import the Required Libraries

Before working with the dataset, import the Python libraries required for data analysis and visualization.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Libraries Used

pandas

Used for working with tables and datasets.

It is normally imported as:

```
import pandas as pd
```

Usage: Use Pandas when you need to load, inspect, filter, clean, or analyze tabular data.

numpy

Used for numerical operations.

```
import numpy as np
```

Usage: NumPy provides numerical functions and is commonly used together with Pandas.

matplotlib.pyplot

Used for creating graphs.

```
import matplotlib.pyplot as plt
```

Usage: Use `plt` to create, label, resize, and display graphs.

seaborn

A visualization library built on Matplotlib.

```
import seaborn as sns
```

Usage: Use Seaborn for statistical graphs such as boxplots, bar plots, and heatmaps.

Part B - Loading the Dataset

Task 2: Load the CSV File

EDA Question

How can we bring the dataset into Python so that we can analyze it?

A CSV file must first be loaded into a Pandas **DataFrame**.

A DataFrame is a table containing rows and columns.

```
df = pd.read_csv("filename.csv")
```

If your CSV file has a different name, replace "Titanic-Dataset.csv" with the correct filename.

For example:

```
df = pd.read_csv("titanic.csv")
```

Function Used: `pd.read_csv()`

`pd.read_csv()` reads a CSV file and converts it into a Pandas DataFrame.

Syntax:

```
pd.read_csv("filename.csv")
```

Usage: Use `pd.read_csv()` to load a CSV file into Python.

Part C - Initial Dataset Inspection

Task 3: Display the First Few Rows

EDA Question

What does the dataset look like?

Before performing analysis, inspect some rows to understand how the data is organized.

```
df.head()
```

By default, `head()` displays the first **5 rows**.

You can also specify the number of rows.

```
df.head(10)
```

Function Used: `head()`

`head()` displays the first rows of a DataFrame.

Usage:

```
df.head()
```

Use it to quickly check column names and sample values.

Student Task

Run `df.head()` and identify:

1. The target column.
 2. Numerical columns.
 3. Categorical or text columns.
-

Task 4: Display Random Records

EDA Question

Do different parts of the dataset contain similar types of records?

`head()` only displays records from the beginning of the dataset. Random records can provide a better general view.

```
df.sample(10)
```

Function Used: `sample()`

`sample()` randomly selects rows from the DataFrame.

Usage:

```
df.sample(10)
```

Use it to display 10 random records.

Student Task

Compare the random rows with the first five rows.

Observe whether there are any missing values such as `NaN`.

Part D - Understanding Dataset Structure

Task 5: Display General Dataset Information

EDA Question

How many columns are present, what are their data types, and which columns contain missing values?

```
df.info()
```

Function Used: `info()`

`info()` displays:

- Column names
- Number of non-null values
- Data types
- Number of rows
- Memory usage

Usage:

```
df.info()
```

Use it when you need a quick technical summary of the dataset.

Important Data Types

You may see data types such as:

- `int64` - integer values
- `float64` - decimal or numerical values
- `object` - usually text or categorical data

Student Questions

After running `df.info()`, answer:

1. How many columns are present?
 2. Which columns contain numerical data?
 3. Which columns contain text or categorical data?
 4. Which columns appear to contain missing values?
-

Task 6: Find the Number of Rows and Columns

EDA Question

What is the size of our dataset?

```
df.shape
```

The result is shown as:

```
(rows, columns)
```

For example:

```
(891, 12)
```

means there are:

- 891 rows
- 12 columns

Attribute Used: `shape`

`shape` returns the dimensions of the DataFrame.

Usage:

```
df.shape
```

Use it to find the number of rows and columns.

Student Task

Write:

Number of rows = _____

Number of columns = _____

Task 7: Check the Data Type of Every Column

EDA Question

What type of data is stored in each column?

```
df.dtypes
```

Attribute Used: `dtypes`

`dtypes` shows the data type of every column.

Usage:

```
df.dtypes
```

Use it to determine whether a column contains integers, decimal values, or text.

Student Question

Which columns are:

- Integer?
 - Float?
 - Object or text?
-

Part E - Missing and Duplicate Data

Task 8: Count Non-Missing Values

EDA Question

How many valid values are available in each column?

```
df.notnull().sum()
```

Functions Used

`notnull()`

Returns `True` where a value exists and `False` where it is missing.

```
df.notnull()
```

`sum()`

Counts the `True` values column by column.

```
df.notnull().sum()
```

Usage: Use `df.notnull().sum()` to count available or non-missing values in every column.

Task 9: Check Duplicate Rows

EDA Question

Does the dataset contain exactly repeated records?

Duplicate rows can affect analysis because the same observation may be counted more than once.

```
df.duplicated().sum()
```

You can print the result clearly:

```
print("Duplicate Rows:", df.duplicated().sum())
```

Functions Used

duplicated()

Checks whether a row is repeated.

`df.duplicated()`

sum()

Counts the duplicate rows.

`df.duplicated().sum()`

Usage: Use `df.duplicated().sum()` to count duplicate records.

Student Question

How many duplicate rows are present?

Duplicate rows = _____

Part F - Understanding Unique Values

Task 10: Check the Values of the Target Variable

EDA Question

What values are used to represent survival?

```
df["Survived"].unique()
```

Function Used: `unique()`

`unique()` returns the different values that appear in one column.

Usage:

```
df["column_name"].unique()
```

Use it to see which distinct values are stored in a column.

For the Titanic dataset:

```
df["Survived"].unique()
```

should help you verify that survival is represented using 0 and 1.

Student Question

What do the values 0 and 1 represent?

Task 11: Count Unique Values in Every Column

EDA Question

How many different values are present in each feature?

```
df.nunique()
```

Function Used: `nunique()`

`nunique()` counts the number of different values in each column.

Usage:

```
df.nunique()
```

Use it to identify columns with few categories or many unique values.

Interpretation

For example:

- Sex may have only 2 unique values.
- Pclass may have 3 unique values.
- PassengerId should contain many unique values.

Student Questions

1. Which columns contain very few unique values?
 2. Which columns appear to work as identifiers?
 3. Which columns may be categorical?
-

Part G - Statistical Summary

Task 12: Generate Descriptive Statistics

EDA Question

What are the basic statistical properties of the numerical columns?

```
df.describe()
```

Function Used: describe()

`describe()` calculates statistics for numerical columns.

It normally includes:

- `count` - number of available values
- `mean` - average
- `std` - standard deviation
- `min` - smallest value
- 25% - first quartile
- 50% - median
- 75% - third quartile
- `max` - largest value

Usage:

`df.describe()`

Use it to get a statistical overview of numerical features.

Student Questions

Using the output of `describe()`, find:

1. Average passenger age.
2. Minimum passenger age.
3. Maximum passenger age.
4. Average fare.
5. Maximum fare.
6. Median age.

Observation

Do not simply copy the complete output table.

Write two or three meaningful observations.

Example format:

The average age of passengers is approximately _____ years.

Passenger fares range from _____ to _____.

The large difference between the median and maximum Fare suggests that some passengers paid much higher fares than most passengers.

Part H - Detailed Missing-Value Analysis

Task 13: Count Missing Values

EDA Question

Which columns contain missing data, and how many values are missing?

```
df.isnull().sum()
```

Functions Used

`isnull()`

Checks every cell for missing data.

```
df.isnull()
```

A missing value returns `True`.

`sum()`

Counts missing values.

```
df.isnull().sum()
```

Usage: Use `df.isnull().sum()` to count missing values in every column.

Task 14: Display Only Columns with Missing Values

Instead of displaying columns that have zero missing values, we can show only problematic columns.

```
missing_values = df.isnull().sum()
```

```
missing_values[missing_values > 0]
```

To sort them from highest to lowest:

```
missing_values[missing_values > 0].sort_values(ascending=False)
```

Function Used: `sort_values()`

`sort_values()` sorts values.

```
.sort_values(ascending=False)
```

`ascending=False` means largest to smallest.

Usage: Use `sort_values(ascending=False)` when you want the highest values first.

Student Question

Which column contains the largest number of missing values?

Task 15: Calculate Missing-Value Percentage

EDA Question

What percentage of each column is missing?

The number of missing values alone may not tell us how serious the problem is. Calculating the percentage makes comparison easier.

```
missing_percentage = (df.isnull().sum() / len(df) * 100).round(2)
```

```
pd.DataFrame({
    "Missing Count": df.isnull().sum(),
    "Missing Percentage": missing_percentage
})
```

Functions Used

`len(df)`

Returns the number of rows.

`len(df)`

Usage: Use `len(df)` to get the total number of observations.

`round(2)`

Rounds numerical values to two decimal places.

`value.round(2)`

Usage: Use `.round(2)` to display numerical results with two decimal places.

`pd.DataFrame()`

Creates a new table or DataFrame.

```
pd.DataFrame({...})
```

Usage: Use `pd.DataFrame()` when you want to organize calculated results into a table.

Student Questions

1. Which column has the highest missing percentage?
2. Is the missing percentage in **Age** small or significant?

3. Does **Embarked** contain many missing values?
 4. Would filling **Cabin** be reliable if most values are missing? Explain briefly.
-

Part I - Data Cleaning

Task 16: Remove Duplicate Rows

If duplicate rows exist, remove them.

```
df = df.drop_duplicates().copy()
```

Function Used: `drop_duplicates()`

Removes duplicate records.

Usage:

```
df.drop_duplicates()
```

Use it when repeated rows should not be included in analysis.

Function Used: `copy()`

`.copy()` creates an independent copy of the resulting DataFrame.

```
df = df.drop_duplicates().copy()
```

Usage: Use `.copy()` when you want to create a separate copy of a DataFrame before making further changes.

Task 17: Handle Missing Age Values

EDA Question

How should missing numerical values in Age be handled?

One common approach is to replace missing ages with the **median age**.

First calculate the median:

```
df["Age"].median()
```

Then fill missing values:

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Function Used: median()

median() returns the middle value after sorting the data.

```
df["Age"].median()
```

Usage: Use median() to calculate the middle value of a numerical feature.

Function Used: fillna()

fillna() replaces missing values.

```
df["Age"].fillna(value)
```

Usage: Use fillna() when missing values should be replaced with another value.

Why Median?

The median is often useful when numerical data contains extreme values because it is less affected by outliers than the mean.

Task 18: Handle Missing Embarked Values

EDA Question

How can missing categorical values be filled?

Embarked is categorical, so we can use its most common value.

Find the most common value:

```
df["Embarked"].mode()
```

Fill the missing values:

```
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

Function Used: mode()

mode() returns the most frequently occurring value.

```
df["column"].mode()[0]
```

Usage: Use mode() to obtain the most common value in a categorical column.

Why [0]?

mode() can return more than one result.

[0] selects the first mode.

Task 19: Verify Missing Values After Cleaning

EDA Question

Did our cleaning operations work correctly?

Always verify the dataset after cleaning.

```
df.isnull().sum()
```

Student Questions

1. Does **Age** still contain missing values?
2. Does **Embarked** still contain missing values?
3. Which column still contains a large number of missing values?

Important Note About Cabin

The **Cabin** column contains a very large amount of missing information in this dataset. Do not automatically replace these values with a random cabin number.

You may leave **Cabin** unchanged.

Part J - Group-Level Analysis

Task 20: Calculate the Overall Survival Rate

EDA Question

What percentage of Titanic passengers survived?

Because:

- 0 means not survived
- 1 means survived

the mean of the **Survived** column gives the proportion of survivors.

```
survival_rate = df["Survived"].mean() * 100

print(f"Overall Survival Rate: {survival_rate:.2f}%")
```

Function Used: `mean()`

Calculates the arithmetic average.

```
df["column"].mean()
```

Usage: Use `mean()` to calculate the average of a numerical column.

Why Does Mean Work Here?

Suppose the values are:

1, 0, 1, 0, 1

There are 3 survivors out of 5 passengers.

The mean is:

$3 / 5 = 0.60$

Multiplying by 100 gives:

60%

Student Task

Write:

Overall survival rate = _____ %

Task 21: Compare Survival Rate by Sex

EDA Question

Did male and female passengers have similar survival rates?

```
df.groupby("Sex")["Survived"].mean() * 100
```

Function Used: `groupby()`

`groupby()` divides data into groups based on a column.

```
df.groupby("Sex")
```

Then:

```
["Survived"].mean()
```

calculates the average survival value for each group.

Usage:

```
df.groupby("group_column")["numeric_column"].mean()
```

Use it to calculate statistics separately for different categories.

Student Questions

1. What was the survival rate for females?
2. What was the survival rate for males?
3. Which group had the higher survival rate?
4. Is the difference small or large?

Task 22: Compare Survival Rate by Passenger Class

EDA Question

Was passenger class related to survival?

```
df.groupby("Pclass")["Survived"].mean() * 100
```

Student Questions

Find the survival rate for:

1st Class = _____ %
2nd Class = _____ %
3rd Class = _____ %

Then answer:

Which passenger class had the highest survival rate?

Part K - Data Visualization

Task 23: Create Histograms for Numerical Features

EDA Question

How are numerical values distributed?

A histogram groups numerical values into ranges and shows how frequently values occur.

Select the numerical columns:

```
numeric_cols = ["Age", "Fare", "SibSp", "Parch"]
```

Create the histograms:

```
df[numeric_cols].hist(  
    bins=30,  
    figsize=(10, 6),  
    layout=(2, 2)  
)
```

```
plt.tight_layout()  
plt.show()
```

Functions and Parameters Used

hist()

Creates histograms.

```
df["Age"].hist()
```

Usage: Use `hist()` to study the distribution of numerical values.

bins

Controls how many intervals the data is divided into.

```
bins=30
```

Usage: Use `bins` to control the number of groups used to display numerical values in a histogram.

More bins produce more detailed intervals.

figsize

Controls graph size.

```
figsize=(10, 6)
```

Usage: Use `figsize` to control the width and height of a graph.

plt.tight_layout()

Adjusts spacing so graphs and labels do not overlap.

```
plt.tight_layout()
```

Usage: Use `plt.tight_layout()` when multiple plots or labels are too close to each other.

plt.show()

Displays the graph.

```
plt.show()
```

Usage: Use `plt.show()` to display the completed visualization.

Student Questions

From the histograms, describe:

1. The general age distribution.
2. Whether Fare values are evenly distributed.
3. Whether most passengers travelled with many or few family members.
4. Which numerical variable appears strongly skewed.

Part L - Univariate Analysis

Task 24: Detect Possible Outliers Using Boxplots

EDA Question

Do numerical features contain unusually high or low values?

A boxplot is useful for finding the spread of data and possible **outliers**.

```
numerical_cols = ["Age", "Fare", "SibSp", "Parch"]
```

```
plt.figure(figsize=(8, 4))
```

```
sns.boxplot(data=df[numerical_cols])
```

```
plt.title("Boxplots of Numerical Features")
```

```
plt.show()
```

Function Used: `sns.boxplot()`

Creates a boxplot.

```
sns.boxplot(data=df[columns])
```

Usage: Use a boxplot to examine spread, median, and possible outliers.

What Is an Outlier?

An outlier is a value that is unusually far from most other observations.

For example, if most passengers paid relatively small fares but a few paid extremely high fares, those high values may appear as outliers.

Student Questions

1. Which feature has the most obvious outliers?
 2. Does **Fare** contain unusually high observations?
 3. Does seeing an outlier automatically mean the value is incorrect? Explain briefly.
-

Part M - Correlation Analysis

Task 25: Create a Correlation Matrix

EDA Question

Are numerical variables related to one another?

Select numerical features:

```
selected_cols = ["Age", "SibSp", "Parch", "Pclass", "Fare"]
```

Calculate correlation:

```
df[selected_cols].corr()
```

Function Used: `corr()`

Calculates correlation between numerical variables.

```
df[numerical_columns].corr()
```

Usage: Use `corr()` to measure the direction and strength of relationships between numerical variables.

Understanding Correlation

Correlation values usually range from:

-1 to +1

- Close to +1 means a strong positive relationship.
 - Close to -1 means a strong negative relationship.
 - Close to 0 means a weak or no linear relationship.
-

Task 26: Display Correlation as a Heatmap

EDA Question

How can we visually compare correlations between numerical variables?

A correlation table can be difficult to read. A heatmap makes the relationships easier to compare visually.

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(  
    df[["Age", "SibSp", "Parch", "Pclass", "Fare"]].corr(),  
    annot=True,  
    cmap="coolwarm",
```

```

        fmt=".2f"
    )

plt.title("Correlation Heatmap")
plt.show()

```

Function Used: `sns.heatmap()`

Displays matrix values using colors.

```
sns.heatmap(correlation_matrix, annot=True)
```

Usage: Use `sns.heatmap()` to visually inspect values in a correlation matrix.

Parameters

`annot=True`

Displays the correlation number inside every cell.

Usage: Use `annot=True` when you want the numerical values to appear inside heatmap cells.

`fmt=".2f"`

Displays numbers with two decimal places.

Usage: Use `fmt=".2f"` to format heatmap values to two decimal places.

`cmap`

Controls the heatmap color scheme.

```
cmap="coolwarm"
```

Usage: Use `cmap` to select the color scheme used in a heatmap.

Student Questions

1. Which two features show the strongest positive correlation?
2. Which variables show a negative correlation?
3. What relationship do you observe between **Pclass** and **Fare**?
4. Does correlation prove that one variable causes another?

Part N - Survival Visualization

Task 27: Compare Survival by Sex Using a Bar Plot

EDA Question

Can we visually compare male and female survival rates?

```
sns.barplot(  
    x="Sex",  
    y="Survived",  
    data=df,  
    hue="Sex",  
    legend=False  
)  
  
plt.title("Sex vs Survival")  
plt.xlabel("Sex")  
plt.ylabel("Average Survival Rate")  
plt.show()
```

Function Used: `sns.barplot()`

Creates a bar chart for categories.

In this case, Seaborn calculates the average `Survived` value for each sex.

```
sns.barplot(x="category", y="numeric_value", data=df)
```

Usage: Use `sns.barplot()` to compare a numerical measurement between categories.

Other Plot Functions

```
plt.title()
```

Adds a title.

```
plt.title("My Graph")
```

Usage: Use `plt.title()` to add a descriptive title to a graph.

```
plt.xlabel()
```

Adds a label to the x-axis.

```
plt.xlabel("Sex")
```

Usage: Use `plt.xlabel()` to describe what the horizontal axis represents.

```
plt.ylabel()
```

Adds a label to the y-axis.

```
plt.ylabel("Survival Rate")
```

Usage: Use `plt.ylabel()` to describe what the vertical axis represents.

Student Question

What conclusion can you make about sex and passenger survival?

Write your conclusion in **2 to 3 sentences**.

Part O - Sorting and Finding Extreme Values

Task 28: Find the Five Highest-Paying Passengers

EDA Question

Which passengers paid the highest fares?

First sort the dataset according to Fare.

```
topFares = df.sort_values(  
    by="Fare",  
    ascending=False  
)
```

Now select the first five records.

```
top5 = topFares.head(5)
```

Display useful columns:

```
top5[["Name", "Sex", "Pclass", "Fare"]]
```

Function Used: `sort_values()`

Sorts the DataFrame according to a column.

```
df.sort_values(by="Fare", ascending=False)
```

Usage: Use `sort_values()` to arrange records from smallest to largest or largest to smallest.

What Does `ascending=False` Mean?

```
ascending=False
```

means sort from the **largest value to the smallest value**.

Task 29: Visualize the Five Highest-Paying Passengers

EDA Question

How can we visually compare the fares of the five highest-paying passengers?

```
sns.barplot(  
    x="Fare",  
    y="Name",  
    data=top5,  
    hue="Sex"  
)  
  
plt.title("Five Highest-Paying Passengers")  
plt.xlabel("Fare")  
plt.ylabel("Passenger Name")  
plt.show()
```

Student Questions

1. Who paid the highest fare?
 2. What passenger class did the highest-paying passengers belong to?
 3. Are their fares similar to the typical fare observed earlier?
-

Part P - Final EDA Questions

After completing all tasks, answer the following questions in your lab report.

Question 1

How many rows and columns are present in the dataset?

Question 2

Which columns contain missing values?

Question 3

Which column contains the highest percentage of missing values?

Question 4

How were missing values in **Age** handled?

Explain why the median was used.

Question 5

How were missing values in **Embarked** handled?

Explain why the mode is suitable for this feature.

Question 6

Were duplicate rows found in the dataset?

Question 7

What is the overall survival rate?

Question 8

Which sex had the higher survival rate?

Question 9

Which passenger class had the highest survival rate?

Question 10

Which numerical feature shows the clearest extreme or outlier values?

Question 11

What do the Fare histograms and boxplots tell you about the distribution of ticket prices?

Question 12

What relationship can you observe between passenger class and fare?

Question 13

Who were the five highest-paying passengers?

Question 14

Based on your complete EDA, mention **three important findings** from the Titanic dataset.

Part Q - Function Quick Reference

Use this section as a quick reminder during the lab.

Function or Attribute	Purpose	Usage
<code>pd.read_csv()</code>	Load CSV data	<code>df = pd.read_csv("file.csv")</code>
<code>df.head()</code>	Display first rows	<code>df.head()</code>
<code>df.sample()</code>	Display random rows	<code>df.sample(10)</code>
<code>df.info()</code>	Display dataset information	<code>df.info()</code>
<code>df.shape</code>	Find rows and columns	<code>df.shape</code>
<code>df.dtypes</code>	Check column data types	<code>df.dtypes</code>
<code>df.notnull()</code>	Check available values	<code>df.notnull().sum()</code>
<code>df.isnull()</code>	Check missing values	<code>df.isnull().sum()</code>
<code>df.duplicated()</code>	Find duplicate rows	<code>df.duplicated().sum()</code>
<code>df.nunique()</code>	Count unique values	<code>df.nunique()</code>
<code>.unique()</code>	Show unique values	<code>df["Sex"].unique()</code>
<code>df.describe()</code>	Generate statistical summary	<code>df.describe()</code>
<code>.mean()</code>	Calculate average	<code>df["Age"].mean()</code>
<code>.median()</code>	Calculate median	<code>df["Age"].median()</code>
<code>.mode()</code>	Find most common value	<code>df["Embarked"].mode()[0]</code>
<code>.fillna()</code>	Fill missing values	<code>df["Age"].fillna(value)</code>
<code>.drop_duplicates()</code>	Remove duplicates	<code>df.drop_duplicates()</code>
<code>.copy()</code>	Create an independent copy	<code>df.copy()</code>
<code>len()</code>	Count rows	<code>len(df)</code>
<code>.groupby()</code>	Analyze groups	<code>df.groupby("Sex")["Survived"].mean()</code>
<code>.sort_values()</code>	Sort records	<code>df.sort_values("Fare")</code>
<code>.corr()</code>	Calculate correlation	<code>df[cols].corr()</code>
<code>.hist()</code>	Create histogram	<code>df["Age"].hist()</code>
<code>sns.boxplot()</code>	Create boxplot	<code>sns.boxplot(data=df)</code>
<code>sns.barplot()</code>	Create bar plot	<code>sns.barplot(x="Sex", y="Survived", data=df)</code>
<code>sns.heatmap()</code>	Create heatmap	<code>sns.heatmap(df[cols].corr())</code>
<code>plt.figure()</code>	Set graph size	<code>plt.figure(figsize=(8, 6))</code>
<code>plt.title()</code>	Add graph title	<code>plt.title("Title")</code>
<code>plt.xlabel()</code>	Add x-axis label	<code>plt.xlabel("Age")</code>
<code>plt.ylabel()</code>	Add y-axis label	<code>plt.ylabel("Count")</code>
<code>plt.tight_layout()</code>	Adjust plot spacing	<code>plt.tight_layout()</code>
<code>plt.show()</code>	Display graph	<code>plt.show()</code>

Part R - Submission Requirements

Your submission should contain:

1. Python code for every task.
2. Output of the important EDA operations.
3. All required graphs.
4. Answers to the EDA questions.
5. Short observations under each major analysis.
6. A final conclusion containing at least three important findings.

Do not write only code.

For every major task, use the following format:

Task:

What are we investigating?

Code:

Python code used to perform the analysis.

Output:

Result or graph produced by Python.

Observation:

What does the result tell us about the dataset?

Final Conclusion

At the end, write a short conclusion of approximately **5 to 8 lines**.

Your conclusion should discuss:

- Dataset size and structure.
- Missing-data problems.
- Important survival patterns.
- Important numerical distributions or outliers.
- What you learned from the EDA process.

The purpose of EDA is not only to generate Python output. The main goal is to understand the dataset before performing further statistical analysis or building a machine-learning model.